

优秀系统设计： 从约束、不变量到证据

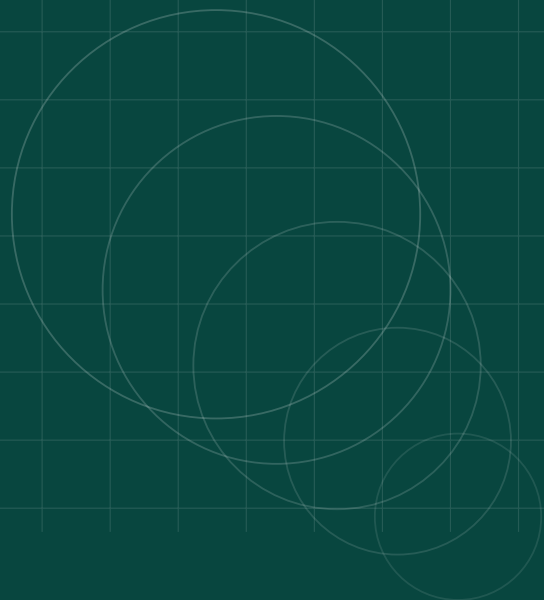
Architecture Review、工程落地与生产准入操作手册

DOCUMENT ESD-REFERENCE-001

VERSION 1.1.0

STATUS BASELINE / OPERATIONAL REFERENCE

OWNER Elliott Bai



文档控制

文档编号	ESD-REFERENCE-001
版本	1.1.0
状态	BASELINE / OPERATIONAL REFERENCE
文档角色	工程参考、评审流程、模板与检查清单
标准套件	优秀系统设计与工程保证标准
发布日期	2026-08-31
Owner	Elliott Bai
复审周期	重大架构变更、关键假设失效、严重事故、风险等级变化，或至少每 12 个月

权威性声明。Markdown 源文件是本册唯一事实来源；本 PDF 是为评审、阅读和归档生成的受控非权威视图。

套件关系与使用方式

本册是标准套件的**工程操作参考**，把 ESD-METHOD-001 转换为可执行的 Stage、Gate、模板、检查清单和完整案例。

本册不重新定义系统设计哲学。原则、术语或语义冲突时以方法论为准；流程可以按风险裁剪，但裁剪必须留下理由、Owner、到期时间和复审条件。

三种阅读模式

快速裁决

从“快速开始”进入各 Stage 的 Gate，直接确认阻塞项。

交付执行

按 Stage 0-9 顺序推进，并复用第 13 章模板。

专项诊断

从完整案例、反模式或 Production Readiness Checklist 反查证据缺口。

版本与变更

本版为 **v1.1.0** 基线，取代 **front matter** 中列出的早期版本。后续任何语义、门禁或证据要求变更，应先更新权威 **Markdown**，再重新生成 **PDF** 与构建清单。

目录

文档控制	i
套件关系与使用方式	ii
版本与变更	iii
快速开始	x
第一章 操作模型	1
1.1 角色	1
1.2 门禁状态	1
1.3 评审原则	2
第二章 风险分级与裁剪	3
2.1 风险等级	3
2.2 裁剪规则	3
2.3 采用剖面	4
2.3.1 Exploration Profile	4
2.3.2 Production-Lite Profile	4
2.3.3 Core Production Profile	4
2.3.4 High-Assurance Profile	5
第三章 Stage 0: Problem Brief 与风险定级	6
3.1 输入	6
3.2 必须产出	6
3.3 关键问题	6
3.4 Gate 0: Problem Validity	6

第四章 Stage 1: Workload、SLO、约束与假设	8
4.1 工作负载最小字段	8
4.2 SLO 与恢复目标	8
4.3 约束与假设登记	8
4.4 Gate 1: Inputs Are Decision-Ready	9
第五章 Stage 2: Critical Properties 与优先级	10
5.1 产出物	10
5.2 编写规则	10
5.3 Gate 2: Semantic Integrity	10
第六章 Stage 3: State、Identity、Ownership 与 Boundary Contract	12
6.1 State Inventory	12
6.2 Boundary Contract	12
6.3 Gate 3: Authority and Composition	13
第七章 Stage 4: Failure、Consistency、Commit 与 Recovery	14
7.1 Failure Matrix	14
7.2 Consistency Decision	14
7.3 Commit Protocol	15
7.4 Gate 4: Failure Semantics Are Closed	15
第八章 Stage 5: Architecture、Planes、Pipeline 与 Modes	16
8.1 架构图最低要求	16
8.2 平面划分	16
8.3 Fast Path / Slow Path	17
8.4 Operating Mode Matrix	17
8.5 Gate 5: Architecture Preserves Semantics	17
第九章 Stage 6: Resource Bounds、Capacity、Economics 与 Control Loops	18
9.1 Resource Budget	18
9.2 Queue Contract	18
9.3 Capacity Worksheet	18

9.4	Economic Model	19
9.5	Control Loop Register	19
9.6	Gate 6: Stable Operating Region	20
第十章	Stage 7: Contracts、Security、Evolution 与 Operations	21
10.1	Contract Checklist	21
10.2	Security Checklist	21
10.3	Evolution Checklist	21
10.4	Operations Checklist	22
10.5	Gate 7: Day-2 Ready	22
第十一章	Stage 8: Verification、Evidence 与 Change Envelope	23
11.1	Assurance Matrix	23
11.2	Evidence Binding	23
11.3	Assurance Meta-invariants	23
11.4	Change Envelope	24
11.5	Gate 8: Evidence Is Decision-Grade	24
第十二章	Stage 9: Production Readiness 与持续保证	25
12.1	Gate 9: Production Admission	25
12.2	放行后的持续保证	25
12.3	Incident → Assurance Delta	26
第十三章	可复用工程模板	27
13.1	One-page System Design Brief	27
13.2	Constraint / Assumption Register	28
13.3	Critical Property Contract	28
13.4	Property Precedence Matrix	28
13.5	State Ownership Register	28
13.6	Boundary Contract Register	29
13.7	Failure / Recovery Matrix	29
13.8	Operating Mode Matrix	29
13.9	Queue / Resource Contract	29

13.10 Capacity & Economic Worksheet 29

13.11 Control Loop Register 30

13.12 Verification & Evidence Matrix 30

13.13 Change Envelope 30

13.14 Exception / Risk Acceptance 31

13.15 ADR Template 31

13.16 Production Readiness Decision 32

13.17 Incident → Assurance Delta 33

第十四章 Architecture Review 会议操作34

14.1 会前材料 34

14.2 会议顺序 34

14.3 Reviewer 应避免 34

14.4 评审记录 35

第十五章 常见反模式与校正36

15.1 Resume-driven Architecture 36

15.2 Architecture Diagram = Architecture 36

15.3 Microservice-first 36

15.4 Unbounded Async 36

15.5 Exactly-once Magic 36

15.6 Retry Everything 37

15.7 Shared Mutable Truth 37

15.8 Monitoring After Launch 37

15.9 Rollback Fantasy 37

15.10 Evidence Illusion 37

15.11 Unknown Becomes Default Continue 37

15.12 Gate Bureaucracy 38

第十六章 完整案例：对象存储 CAN 分块文件处理流水线39

16.1 场景、风险等级与 Gate 映射 39

16.1.1 风险等级 39

16.1.2 Gate 映射	39
16.2 Goals / Non-goals	40
16.2.1 Goals	40
16.2.2 Non-goals	40
16.3 Workload、SLO、约束与假设	40
16.3.1 工作负载模型	40
16.3.2 初始 SLO 示例	41
16.3.3 关键约束	41
16.3.4 关键假设登记示例	41
16.4 关键性质	42
16.4.1 CP-01 — 单一正式完成结果	42
16.4.2 CP-02 — DBC 可追踪且不猜测	42
16.4.3 CP-03 — Partial output 不可见为正式完成	43
16.4.4 CP-04 — 资源有界	43
16.4.5 CP-05 — 已接纳任务有显式终态	43
16.5 状态与所有权	43
16.5.1 稳定身份	44
16.6 DBC 绑定语义	44
16.7 推荐逻辑架构	45
16.8 为什么默认避免“两遍偏移预扫描”	45
16.9 下载与缓冲策略	46
16.10 CPU 调度与局部性	46
16.11 幂等与提交协议	47
16.12 有界资源示例	48
16.13 容量模型	48
16.13.1 故障降容与积压恢复	49
16.13.2 单位经济性	49
16.14 失败与恢复矩阵节选	49
16.15 关键可观测性	50
16.16 验证计划	50

16.17 Rust 与 C++23 的正确决策位置	51
16.18 Change Envelope 与示例裁决	51
第十七章 Production Readiness Checklist	53
17.1 问题与语义	53
17.2 状态与边界	53
17.3 失败与恢复	53
17.4 资源、性能与成本	54
17.5 安全、运维与演进	54
17.6 验证与证据	54
第十八章 最小 19 问设计清单	55
第十九章 团队成熟度与采用路线	56
第二十章 文档与标准维护	57
20.1 版本规则	57
20.2 复审触发	57
20.3 单一事实来源	57
20.4 两篇文档的最终边界	58
20.5 结论	58

Architecture Review、工程落地与生产准入操作手册

文档属性	内容
文档编号	ESD-REFERENCE-001
版本	1.1.0
状态	BASELINE / OPERATIONAL REFERENCE
规范来源	ESD-METHOD-001 《优秀系统设计与工程保证方法论》
核心用途	系统设计、Architecture Review、ADR、容量规划、生产准入、事故后保证增量
适用对象	设计作者、Reviewer、技术负责人、平台/安全/SRE、发布裁决者
权威格式	本 Markdown 文件；其他导出不是独立事实来源

规范来源：ESD-METHOD-001 《优秀系统设计与工程保证方法论》

使用规则

本文不重新定义系统设计哲学，而是把 ESD-METHOD-001 转换为工程阶段、产出物、门禁、模板与案例。原则、术语或语义发生冲突时，以方法论文档为准；本文中的默认流程可按风险裁剪，但裁剪必须留下理由、Owner、到期时间和复审条件。

快速开始

对一个新系统或重大变更，最小 workflow 是：

0. 定义问题、目标、非目标和风险等级

1. 建立工作负载、SLO、约束与假设

2. 登记关键性质及其优先级

3. 明确状态、身份、所有权和边界契约

4. 建模失败、一致性、提交与恢复

5. 推导架构、平面、流水线和运行模式

6. 计算资源上界、容量、成本与控制回路

7. 设计契约、安全、演进、迁移与操作

8. 建立验证、证据和变更暴露计划

9. 通过生产准入，并持续监测假设与证据漂移

任何阶段出现以下情况，都不得以“后续补充”为由自动通过：

- 关键状态没有权威 Owner；
- 高后果性质没有精确作用域；
- 重复、部分成功或 ACK 顺序不明确；

- 关键资源无上界；
- 生产完成语义和恢复权威不明确；
- 关键证据无法绑定当前制品、配置或环境；
- 关键假设无 **Owner**、无监测或已失效；
- 高风险例外没有范围和到期时间。

操作模型

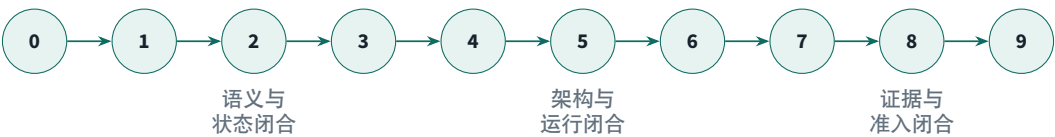


图 1 Stage 0-9: 每个阶段均以可裁决 Gate 收敛

1.1 角色

角色	责任
Design Author	维护问题、假设、性质、方案、风险与证据链
Semantic Owner	定义业务语义、状态身份、契约与兼容规则
Operational Owner	负责 SLO、容量、发布、恢复、on-call 与事故响应
Security / Compliance Reviewer	审查信任边界、权限、审计和硬约束
Platform / SRE Reviewer	审查资源、故障、可观测性、发布和恢复
Assurance Reviewer	检查主张、验证、证据覆盖和认识边界
Decision Authority	根据风险、证据和例外做最终门禁裁决

高风险变更中，提出方案、评价证据和最终批准不得由同一非受控主体兼任。

1.2 门禁状态

状态	含义
GO	强制条件满足，证据与当前对象绑定，剩余风险在阈值内
CONDITIONAL_GO	可在明确范围、期限、Owner、监测和撤销条件下放行
NO_GO	已知违反关键要求，或风险不可接受
UNKNOWN	证据不足、已过期、无法复现、作用域不清或关键假设未验证

状态	含义
NOT_APPLICABLE	经论证与记录后确实不适用

UNKNOWN 不是较弱的 **GO**。对高风险性质，**UNKNOWN** 默认阻断生产放行。

1.3 评审原则

评审顺序必须从语义到机制：

Problem -> Property -> State -> Failure -> Resource -> Architecture -> Technology

不应从“为什么不用某组件”开始，也不应让架构图的视觉完整性掩盖状态和失败语义缺失。

风险分级与裁剪

2.1 风险等级

等级	典型场景	后果	默认保证深度
R0 — Exploration	一次性实验、无真实敏感数据、可随时删除	局部、可逆	最小 Problem Brief、数据边界、退出条件
R1 — Bounded Production	有限用户或内部系统，影响可隔离	中等、可恢复	关键性质、Owner、失败语义、资源上界、基础证据
R2 — Core Production	核心数据、收入、跨团队依赖、持续运行	高、恢复昂贵	完整阶段与门禁、容量/恢复模型、运行模式、证据绑定
R3 — High Assurance	资金、安全、合规、人身风险或大爆炸半径	极高或不可逆	独立审批、组合保证、元不变量、例外治理、持续漂移检测

风险等级必须根据**后果**、**暴露半径**、**不确定性**和**不可逆性**确定，不能只根据项目规模或 QPS。

2.2 裁剪规则

允许裁剪执行材料，但不得裁剪认识边界。任何裁剪必须记录：

omitted requirement
reason
risk level
compensating control
owner
expiry / review trigger

以下内容即使在 MVP 中也通常不可省略：

- 核心数据身份与权威来源；
- 不可逆外部副作用；
- 认证、授权和租户隔离；
- 外部长期契约；
- 资源硬上限与删除/退出路径；
- 无法回滚的数据或 Schema 变化。

2.3 采用剖面

2.3.1 Exploration Profile

Problem / Non-goals
Data and trust boundary
Irreversible decisions
Hard resource cap
Exit / deletion criteria

2.3.2 Production-Lite Profile

在 Exploration 基础上增加：

Critical Property Register
State Ownership
Failure / Recovery Matrix
Basic capacity estimate
Operational Owner
Smoke / contract / fault evidence

2.3.3 Core Production Profile

增加：

Boundary Contracts
Operating Modes
Capacity and recovery model
Change Envelope
Evidence binding and freshness
Production Readiness Gate

2.3.4 High-Assurance Profile

增加：

- Property precedence
- Independent reviewer / approver
- Assurance meta-invariants
- Exception and break-glass governance
- Evidence drift automation
- Assurance delta after incidents

Stage 0: Problem Brief 与风险定级

3.1 输入

- 业务背景、当前问题或机会；
- 主要参与者和决策主体；
- 当前系统事实与已知事故；
- 交付窗口和初始预算。

3.2 必须产出

```
Problem statement
Goals
Non-goals
Success metrics
Failure consequences
Risk tier
Decision deadline
Initial unknowns
```

3.3 关键问题

1. 解决的是业务问题、运营问题还是技术便利？
2. 不做什么，防止哪些范围膨胀？
3. 失败会造成延迟、数据错误、资金损失、安全问题还是不可恢复损坏？
4. 什么结果可以客观证明项目成功？
5. 这是系统新建、迁移、扩容、可靠性修复还是成本优化？

3.4 Gate 0: Problem Validity

GO 条件：

- 问题、价值和目标清楚；
- 成功标准可检验；
- 风险等级有依据；
- 不以预选技术作为问题定义。

典型 **NO_GO**：

- “因为要上 Kubernetes/微服务/AI”；
- 问题无法与任何业务或运营后果关联；
- 不知道失败影响却直接承诺架构。

Stage 1: Workload、SLO、约束与假设

4.1 工作负载最小字段

维度	内容
Arrival	平均、峰值、突发时长、增长率
Size	item/byte 分布、p99、硬上限
Work	CPU、I/O、网络、压缩、fan-out、外部调用
Shape	热点、租户偏斜、读写比、顺序、重复、迟到
Modes	正常、峰值、恢复、重放、依赖降容
Retention	保留期、重建时间、存储增长

4.2 SLO 与恢复目标

必须分别记录：

```
availability
latency / completion time
freshness / lag
correctness or error budget where meaningful
RPO
RTO
recovery catch-up objective
capacity headroom target
```

正确性硬约束不得藏在 error budget 中。

4.3 约束与假设登记

约束至少分为：业务、技术、组织、经济、安全/合规和时间。假设必须可证伪，并关联 Owner、证据、监测信号、失效阈值和响应。

4.4 Gate 1: Inputs Are Decision-Ready

GO 条件:

- 已覆盖真实峰值和恢复负载;
- 工作负载单位明确;
- SLO 与硬正确性分离;
- 团队技能、预算和运维约束已进入模型;
- 关键假设可监测且有复审条件。

UNKNOWN 条件:

- 只有平均值, 没有尾部和硬上限;
- 无法估计恢复积压;
- 成本数据缺失且连粗估范围都没有;
- 关键外部依赖语义未确认。

Stage 2: Critical Properties 与优先级

5.1 产出物

- Critical Property Register;
- Property Precedence Matrix;
- Safety、Liveness、Durability、SLO、Assumption 和 Policy 分类;
- 每项性质的违反后果和验证义务。

5.2 编写规则

弱表述:

系统应可靠、幂等、高可用。

强表述:

对相同 `source_object_version` 与 `pipeline_semantic_version`,
`Result Store` 在任意可观察时刻最多存在一个 `committed completion manifest`;
冲突执行只能返回 `ALREADY_COMMITTED` 或进入 `reconciliation`, 不能覆盖正式结果。

每项关键性质必须包含: 身份、作用域、时间语义、假设、保证、允许降级、权威边界、Owner、验证覆盖和证据失效规则。

5.3 Gate 2: Semantic Integrity

GO 条件:

- 高后果风险均有对应关键性质;
- 性质可判定, 不是愿望;
- 性质冲突已有优先级;

- **UNKNOWN** 和 allowed degradation 被明确；
- 每项性质拥有 Owner 和验证义务。

典型 **NO_GO**：

- “exactly once” 没有定义作用域；
- 可用性在事故现场可以任意覆盖数据完整性；
- 对未知输入默认继续执行。

Stage 3: State、Identity、Ownership 与 Boundary Contract

6.1 State Inventory

对每份状态记录：

```
state name
business meaning
stable identity
authoritative / derived / ephemeral / evidence / configuration
authoritative store
mutation authority
technical owner
operational owner
security owner
retention
recovery source
conflict semantics
```

6.2 Boundary Contract

对关键边界记录：

```
Assumptions
Guarantees
Violation behavior
Identity and version
Timeout / retry / ordering / duplicate semantics
Capacity assumptions
Verification
Owner
```

6.3 Gate 3: Authority and Composition

GO 条件：

- 每份权威状态唯一；
- 复制、缓存和派生状态不会伪装成多权威；
- 技术与运营 Owner 明确；
- 上游 Guarantee 能满足下游 Assumption；
- 违反边界契约时行为显式、有界、可观测；
- 跨团队边界与 Bounded Context 基本对齐。

典型 **NO_GO**：

- 多个服务直接修改同一业务表但无人裁决；
- 依赖本地内存恢复关键事实；
- 服务 API 只定义成功响应，不定义重复、冲突和超时。

Stage 4: Failure、Consistency、Commit 与 Recovery

7.1 Failure Matrix

至少覆盖：

- 请求或消息重复；
- 乱序、迟到和部分可见；
- 网络 timeout 与请求结果未知；
- worker crash、进程重启和 consumer rebalance；
- 写入成功但 ACK 失败；
- ACK 成功但业务提交失败；
- 依赖变慢、不可用或返回陈旧数据；
- 磁盘写满、内存耗尽、连接池耗尽；
- 配置错误、版本不兼容和权限变更；
- 区域或控制面故障；
- 恢复期间新流量与积压竞争。

每项失败记录：检测、可能已发生的副作用、权威裁决、自动动作、人工动作、重试安全性、最大影响、恢复目标和证据。

7.2 Consistency Decision

对每个关键状态明确：

```
visibility requirement
ordering requirement
atomicity boundary
isolation requirement
conflict behavior
staleness tolerance
```

```
coordination mechanism
availability / latency trade-off
```

7.3 Commit Protocol

必须画出业务完成状态机：

```
DISCOVERED
-> ACCEPTED
-> RUNNING
-> OUTPUT_STAGED
-> AUTHORITATIVELY_COMMITTED
-> ACKNOWLEDGED

terminal alternatives:
  QUARANTINED
  PERMANENTLY_REJECTED
  RECONCILIATION_REQUIRED
```

只有 `AUTHORITATIVELY_COMMITTED` 或明确业务定义的等价状态才构成正式完成。

7.4 Gate 4: Failure Semantics Are Closed

GO 条件：

- 部分成功和未知结果有明确处理；
- `ACK` 顺序与业务完成语义一致；
- 重试有稳定身份和预算；
- 幂等由原子机制构造，不依赖“通常不会重复”；
- 恢复从权威状态出发；
- 降级、隔离和恢复模式已定义。

典型 **NO_GO**：

- 依赖消息中间件标签宣称端到端 `exactly-once`；
- 超时后一律重试不可逆操作；
- 回滚只描述切换旧镜像，忽略数据已经变化。

Stage 5: Architecture、Planes、Pipeline 与 Modes

8.1 架构图最低要求

架构图必须能够表达：

- 权威状态与派生状态；
- 数据流、控制流、管理流和证据流；
- trust boundary 与 mutation authority；
- sync / async 边界；
- commit、ACK 和可见性边界；
- 背压方向；
- failure domain 与 isolation boundary；
- 正常、降级和恢复路径。

只画服务方框和箭头不构成完整架构。

8.2 平面划分

Data Plane:

高频传输、计算、解码、查询或业务执行

Control Plane:

策略、调度、路由、配置和收敛

Management Plane:

发布、升级、诊断、恢复、人工操作

Assurance Plane:

验证、证据、门禁、例外、漂移判定

平面可以部署在同一进程中，但责任、权限和失败语义必须区分。

8.3 Fast Path / Slow Path

Fast Path 应只处理已验证、常见、可缓存的正常输入。损坏、未知版本、冲突、人工裁决和长期补偿应进入显式 Slow Path 或 Quarantine，而不是把热路径变成无限分支。

8.4 Operating Mode Matrix

至少定义：

Mode	Entry	Hard Properties	Allowed	Forbidden	Exit
			Degradation		
NORMAL	关键依赖健康	全部硬性质	无	猜测、旁路提交	—
DEGRADED	依赖或容量下降	完整性、安全、资源上界	延迟、完成率	将未知结果标正式	依赖恢复并校验积压
RECOVERY	故障后重新收敛	权威状态、可追踪重放	暂停新接纳	用临时状态强行完成	reconciliation 完成
QUARANTINE	输入或语义未知	隔离、审计、不可污染正式结果	活性	自动猜测修复	人工/规则裁决
MAINTENANCE	计划变更	数据和权限边界	可用性	未声明数据迁移	迁移验证完成
EMERGENCY	正常控制面阻断关键恢复	审计、时限、最小权限	正常审批路径	无期限 break-glass	补偿审查完成

8.5 Gate 5: Architecture Preserves Semantics

GO 条件：

- 架构能够逐项解释关键性质如何被维护；
- 平面和权限边界合理；
- 快慢路径与故障隔离明确；
- 运行模式可判定、可观测、有退出条件；
- 技术组件是推导结果，而不是前置约束的伪装。

Stage 6: Resource Bounds、Capacity、Economics 与 Control Loops

9.1 Resource Budget

必须为以下资源定义上界：

```
inflight work
inflight bytes
queue depth / queued bytes
threads / tasks / permits
connections / file descriptors
cache and temporary disk
retry attempts / retry time
reconciliation backlog
per-tenant share
```

9.2 Queue Contract

每个队列记录：目的、容量单位、容量值、生产/消费速率、满载行为、背压方向、公平性、数据持久性、drain/crash 语义、指标和 Owner。

9.3 Capacity Worksheet

最小计算：

```
input rate
x work per input
= CPU / I/O / network demand

inflight bytes
```

```
= arrival byte rate x residence time

required service rate
>= peak arrival rate + recovery catch-up rate
```

同时评估：

- 单节点与集群上限；
- N-1 或目标故障域降容；
- 扩容和冷启动延迟；
- 恢复积压的清空窗口；
- 外部依赖限额；
- 热点与租户公平；
- 尾延迟和排队时间。

9.4 Economic Model

至少提供 C1 粗估：

```
compute
storage read/write
network
message / request charges
retained evidence
replay / recovery
operational overhead
```

并给出单位成本和置信等级。无法计算时必须标为 **UNKNOWN** ，不能写“成本可接受”。

9.5 Control Loop Register

字段	内容
Controlled Variable	queue age、lag、utilization、error rate 等
Sensor / Delay	信号、窗口、采样和延迟
Controller	admission、autoscaling、retry、routing 等
Actuator	permits、replicas、traffic weight、feature flag
Bounds	动作上下限和速率
Interaction	与其他控制回路的耦合

字段	内容
Safe Default	控制器失败时的默认行为
Verification	仿真、故障注入或历史回放

9.6 Gate 6: Stable Operating Region

- GO 条件：
- 所有可增长资源有界；
 - 背压能够传播到真正可控的上游；
 - 峰值、故障降容和恢复容量满足目标；
 - retry amplification 有上界；
 - 控制回路不存在明显正反馈放大；
 - 单位成本有模型和监测路径；
 - 新增复杂度有 Owner 和回收条件。

- 典型 NO_GO：
- async everywhere + unbounded queue；
 - 只算稳态，不算恢复；
 - autoscaling 被当作无限容量；
 - 每层各自重试且没有共享 deadline。

Stage 7: Contracts、Security、Evolution 与 Operations

10.1 Contract Checklist

- 稳定 identity 与 idempotency key;
- schema 和 semantic version;
- unknown field / unknown enum / malformed input 行为;
- ordering、duplicate、timeout、retry 和 conflict;
- compatibility window;
- retention 与删除语义;
- audit 与 observability;
- deprecation 和 migration path。

10.2 Security Checklist

- trust boundary 和不可信输入;
- human/service/pipeline/recovery identity;
- least privilege 与 mutation authority;
- secret、key 和证书生命周期;
- data classification、encryption 和 retention;
- control plane / assurance plane 变更权限;
- break-glass 范围、时限、审计与复审;
- dependency / artifact provenance。

10.3 Evolution Checklist

- N/N-1 或声明的兼容矩阵;
- rolling upgrade 混合版本行为;
- expand-migrate-contract;

- dual-read / shadow / reconciliation;
- rollback 与 forward-recovery;
- replay 对历史版本的处理;
- ADR 失效条件和复审触发。

10.4 Operations Checklist

- deploy、upgrade、drain、shutdown;
- rollback、forward repair、reconcile;
- backup、restore、RPO、RTO;
- dependency outage 和区域故障 runbook;
- capacity expansion 和 emergency shedding;
- ownership、escalation 与 incident command;
- quarantine 清理和长期积压处理。

10.5 Gate 7: Day-2 Ready

GO 条件:

- 外部契约和版本策略明确;
- 权限与变更权威符合最小化原则;
- 数据变化具有安全迁移路径;
- 回滚不依赖不成立的数据兼容假设;
- 部署、排空、恢复、重放和灾难恢复可执行;
- 运营 Owner 已接受责任。

Stage 8: Verification、Evidence 与 Change Envelope

11.1 Assurance Matrix

每项关键性质建立：

Claim	Mechanism	Verification	Evidence	Coverage	Limitations	Freshness	Owner
-------	-----------	--------------	----------	----------	-------------	-----------	-------

验证应组合：类型/静态检查、单元与属性测试、契约/集成测试、负载、故障注入、恢复演练、策略检查、运行监控和人工审查。

11.2 Evidence Binding

关键证据必须绑定：

```
subject_digest
configuration_digest
environment_identity
workload_corpus_digest
fault_profile
verifier_identity
produced_at
validity / invalidation rule
coverage and limitations
```

11.3 Assurance Meta-invariants

生产流水线必须检查：

- 缺失或过期证据不转为 **PASS** ；

- **required checks**、验证器和豁免规则的修改本身受门禁；
- 高风险审批职责适当分离；
- 证据不可静默修改；
- 保证面不可用时不默认放行；
- 运行漂移会使相关结论失效；
- 例外和 **break-glass** 有范围、到期与补偿复审。

11.4 Change Envelope

重要变更必须定义：

```
semantic delta
assumptions
maximum exposure
blast-radius boundary
representative traffic / corpus
observation window
success criteria
abort conditions
rollback / forward-recovery
owner and decision authority
evidence to retain
```

11.5 Gate 8: Evidence Is Decision-Grade

GO 条件：

- 每项高风险性质有适当验证；
- 证据与当前制品、配置、环境绑定；
- 覆盖和限制被明确记录；
- 性能证据使用代表性负载与统计方法；
- 故障与恢复证据覆盖关键边界；
- **Change Envelope** 有自动或明确的停止条件。

UNKNOWN 条件：

- 只展示测试数量或覆盖率，没有对应性质；
- 性能数据来自不同硬件或不同制品；
- 没有验证重复、崩溃或依赖退化；
- **canary** 没有预定义成功和退出条件。

Stage 9: Production Readiness 与持续保证

12.1 Gate 9: Production Admission

生产放行前必须确认：

```
all mandatory gates resolved
all blocking UNKNOWN resolved or explicitly rejected
residual risks accepted by authority
operational owner ready
runbooks and dashboards available
rollback / forward-recovery viable
change envelope configured
evidence retained and traceable
```

12.2 放行后的持续保证

运行时必须持续观察：

- 工作负载是否越过假设范围；
- 关键依赖、Schema、策略和配置是否漂移；
- 证据是否到期或失效；
- 关键性质对应的语义指标是否异常；
- retry amplification、queue age 和 recovery debt；
- 单位成本和容量 headroom；
- 例外是否到期；
- Assurance Plane 是否可用且未被绕过。

12.3 Incident → Assurance Delta

重大事故、近失事件或长时间 **UNKNOWN** 必须产生至少一项保证增量：

```
new or revised property  
new assumption / boundary contract  
new failure scenario  
new mechanism  
new verification  
new runtime evidence or alert  
new authority / runbook  
revised capacity or economic model  
revised ADR
```

可复用工程模板

13.1 One-page System Design Brief

```
# System / Change Name

## 1. Context and Business Outcome

## 2. Problem

## 3. Goals

## 4. Non-goals

## 5. Risk Tier and Failure Consequences

## 6. Workload and Growth

## 7. SLO / RPO / RTO

## 8. Constraints and Assumptions

## 9. Critical Properties

## 10. State, Identity and Ownership

## 11. Failure / Consistency / Commit

## 12. Architecture and Operating Modes

## 13. Resource Bounds and Capacity

## 14. Security and Trust Boundaries

## 15. Evolution / Migration / Recovery

## 16. Verification and Evidence

## 17. Alternatives and Decision

## 18. Unknowns, Exceptions and Review Triggers
```

13.2 Constraint / Assumption Register

ID	Type	Statement	Depen- dents	Evidence	Signal	Owner	Expiry / In- validation	Response
A-001	Assump- tion							

建 议 状 态 ： PROPOSED / VALIDATED / MONITORED / AT_RISK / INVALIDATED / RETIRED。

13.3 Critical Property Contract

```
### CP-XXX — Property Name

- Category: Safety / Liveness / Durability / Isolation / Recovery / SLO
- Identity:
- Scope:
- Temporal semantics:
- Assumptions:
- Guarantee:
- Allowed degradation:
- Authority boundary:
- Violation consequence:
- Mechanism:
- Verification coverage:
- Evidence freshness:
- Owner:
```

13.4 Property Precedence Matrix

Property A	Property B	Conflict		Allowed		Decision	
		Scenario	Priority	Sacrifice		Mechanism	Authority

13.5 State Ownership Register

State	Meaning	Identity	Class	Authority	Mutation	Technical	Opera- tional	Recovery	Conflict
					Authority	Owner	Owner	Source	

13.6 Boundary Contract Register

Boundary	Assumptions	Guarantees	Violation Behavior	Version	Capacity / Time	Verification	Owner
					Assumption		

13.7 Failure / Recovery Matrix

Failure	Detection	Possible	Authority	Automatic	Retry	Manual	Bound	Evidence
		Partial		Action	Safety	Action		
		Effect						

13.8 Operating Mode Matrix

Mode	Entry / Authority	Hard Properties	Allowed		Forbidden		Max Duration	Evidence
			Degrada- tion	Resource Policy	Exit			

13.9 Queue / Resource Contract

```
### Resource / Queue Name

- Purpose:
- Capacity unit:
- Hard bound:
- Producer rate:
- Consumer rate:
- Full behavior:
- Backpressure direction:
- Fairness / isolation:
- Persistence:
- Drain / crash semantics:
- Metrics and alerts:
- Owner:
```

13.10 Capacity & Economic Worksheet

```
## Workload

- item rate:
- byte rate:
- size distribution:
```

```
- peak and burst:
- growth:

## Per-item Work
- CPU:
- memory residency:
- network:
- storage I/O:
- external calls:

## Stage Capacity
| Stage | Service Time | Parallelism | Capacity | Utilization Target | Queue Bound |

## Failure / Recovery
- N-1 capacity:
- backlog after outage:
- catch-up target:
- scaling delay:

## Economics
- compute:
- storage:
- network:
- managed-service charges:
- replay / recovery:
- evidence retention:
- unit cost:
- confidence level: C0 / C1 / C2 / C3
```

13.11 Control Loop Register

Loop	Variable	Sensor /		Controller	Actuator	Bounds	Interac- tion	Safe Default	Verifica- tion	Owner
		Delay								

13.12 Verification & Evidence Matrix

Property	Mecha- nism	Verifica- tion	Evidence ID	Subject /		Limita- tions	Status	Freshness	Owner
				Config / Env	Coverage				

证据状态：CURRENT / STALE / INVALIDATED / SUPERSEDED / NOT_REPRODUCIBLE。

13.13 Change Envelope

```
| # Change Envelope
```

- Change / semantic delta:
- Assumptions:
- Risk tier:
- Maximum exposure:
- Blast-radius boundary:
- Representative traffic / corpus:
- Observation window:
- Success criteria:
- Abort conditions:
- Rollback / forward-recovery:
- Owner:
- Decision authority:
- Evidence to retain:

13.14 Exception / Risk Acceptance

- ```
Exception EX-XXX
```
- Requirement being bypassed:
  - Scope:
  - Reason:
  - Risk:
  - Compensating control:
  - Owner:
  - Approver:
  - Start:
  - Expiry:
  - Revocation condition:
  - Required follow-up:
  - Evidence / audit link:

无到期时间的例外不得进入正式基线。

## 13.15 ADR Template

- ```
# ADR-NNNN: Decision Title
```
- Status:
 - Date:
 - Owner:
 - Risk tier:
- ```
Context
```
- ```
## Problem and Constraints
```
- ```
Decision
```
- ```
## Assumptions
```

```
## Alternatives Considered

## Consequences

## Risks and Unknowns

## Security and Operational Impact

## Capacity and Cost Impact

## Verification Evidence

## Reversibility

## Migration / Rollback / Forward-Recovery

## Invalidation Conditions

## Review Triggers

## Follow-up Actions
```

13.16 Production Readiness Decision

```
# Production Readiness Decision

- System / change:
- Version / subject digest:
- Environment:
- Risk tier:
- Decision: GO / CONDITIONAL_GO / NO_GO / UNKNOWN
- Decision authority:
- Date:

## Gate Summary
| Gate | State | Evidence / Finding | Owner | Expiry / Follow-up |
|---|---|---|---|---|

## Blocking Unknowns
-

## Accepted Residual Risks
- scope:
- risk owner:
- compensating control:
- expiry / revocation:

## Change Envelope
- maximum exposure:
- observation window:
- abort conditions:
- rollback / forward-recovery:
```

```
## Evidence Package
- property evidence:
- capacity evidence:
- fault / recovery evidence:
- security evidence:
- cost evidence:

## Final Sign-off
- Semantic Owner:
- Operational Owner:
- Security / Compliance:
- Assurance Reviewer:
- Decision Authority:
```

13.17 Incident → Assurance Delta

```
# Incident / Near-miss Assurance Delta

- Incident:
- Original blind spot:
- Affected property / assumption / contract:
- New or revised property:
- Mechanism change:
- Verification change:
- Runtime evidence / alert change:
- Runbook / authority change:
- Capacity / cost change:
- ADR / architecture change:
- Owner:
- Completion evidence:
```

事故复盘只有在改变性质、假设、机制、验证、证据、权限或架构中的至少一项后，才形成了可验证的系统性改进。

Architecture Review 会议操作

14.1 会前材料

Reviewer 应在会前获得：

- One-page Brief;
- 风险等级；
- Critical Property Register；
- State / Ownership 与 Failure Matrix；
- 至少一个备选方案；
- 容量粗算；
- 关键未知和需要会议裁决的问题。

没有这些材料时，会议很容易退化为现场看图和技术偏好辩论。

14.2 会议顺序

推荐顺序：

1. 作者用五分钟说明问题、非目标和风险；
2. 先审关键性质与状态权威；
3. 再审失败、提交、资源和恢复；
4. 然后审架构是否维护这些语义；
5. 最后才讨论技术选型、实现与优化；
6. 明确 GO / CONDITIONAL_GO / NO_GO / UNKNOWN；
7. 每个未决项必须有 Owner、期限和阻断级别。

14.3 Reviewer 应避免

- 用个人熟悉度替代约束分析；
- 用“业界都这样”替代性质证明；
- 在未理解权威状态前建议拆服务；

- 只审正常路径；
- 用更多监控掩盖语义不清；
- 把所有问题都推成“实现细节”；
- 追求文档完整度而忽视真实高风险缺口。

14.4 评审记录

每次评审至少记录：

```
decision state
blocking findings
accepted residual risks
unknowns
exceptions
actions / owners / deadlines
evidence required
review trigger
```

常见反模式与校正

15.1 Resume-driven Architecture

症状：先选 Kafka、Kubernetes、微服务、Rust、Service Mesh，再寻找需求。

风险：复杂度先于价值，状态和失败语义被组件标签掩盖。

校正：要求每个组件映射到具体性质、负载或运营问题；无法映射则默认删除。

15.2 Architecture Diagram = Architecture

症状：架构图精美，但没有状态权威、ACK、失败和恢复。

校正：图中必须标记权威状态、提交边界、背压、信任边界与故障域；并配合 Register。

15.3 Microservice-first

症状：按名词或团队人数拆服务，每个服务共享数据库或同步级联。

风险：形成分布式单体，增加协调和恢复状态。

校正：优先 modular monolith 或清晰 bounded context；只有独立状态、变化率、扩展或故障隔离需求才引入分布式边界。

15.4 Unbounded Async

症状：异步下载、解压、处理和写出之间全部使用无界 channel。

风险：下游变慢时内存增长、超时、重试和级联故障。

校正：按 bytes 或真实成本设 permit，建立端到端背压和满载策略。

15.5 Exactly-once Magic

症状：依赖消息系统或流框架标签声称端到端 exactly-once。

风险：外部副作用、重放和跨系统提交仍会重复。

校正：定义业务 effect 的作用域、稳定身份、原子提交和冲突语义。

15.6 Retry Everything

症状：所有 timeout 和错误都自动重试。

风险：永久错误被放大，依赖故障形成 retry storm。

校正：错误分类、共享 deadline、retry budget、jitter、最大放大比和准入联动。

15.7 Shared Mutable Truth

症状：多个服务或团队直接修改同一权威表。

风险：规则分散、冲突无裁决、迁移不可控。

校正：单一 mutation authority，其他主体通过契约提议变更或消费投影。

15.8 Monitoring After Launch

症状：上线后再补 CPU、内存和错误率。

风险：无法判断系统为什么慢、是否重复、是否使用错误版本。

校正：从性质、队列、状态机和恢复模型推导语义指标。

15.9 Rollback Fantasy

症状：发布计划只写“失败则回滚”。

风险：新数据、Schema 或外部副作用已使旧代码不兼容。

校正：区分 binary rollback、data compatibility、reconciliation 和 forward repair。

15.10 Evidence Illusion

症状：测试很多、覆盖率高、CI 全绿，因此宣称系统正确。

风险：证据没有映射性质，或与当前制品/环境无关。

校正：从 Claim 开始建立 Evidence Matrix，并记录覆盖与限制。

15.11 Unknown Becomes Default Continue

症状：未知版本、未知身份或证据缺失时使用默认值继续。

风险：把认识不足转换为数据污染或越权行为。

校正：根据风险 fail closed、quarantine 或返回显式 **UNKNOWN**。

15.12 Gate Bureaucracy

症状：门禁数量增加，但与风险无关；团队只优化绿色状态。

风险：Goodhart 化、交付停滞、真实风险反而被隐藏。

校正：门禁必须关联关键性质或事故；定期删除低价值检查；按风险裁剪。

完整案例：对象存储 CAN 分块文件处理流水线

本案例用于验证方法的可执行性。数值仅为示例，正式设计必须用真实采样和基准替换。

16.1 场景、风险等级与 Gate 映射

- 文件发现服务将对象元数据写入 Kafka；
- 每条消息包含对象 URI、对象版本、大小、校验信息和采集元数据；
- 单个对象不超过 40 MiB；
- 文件由大量“业务头 + gzip member”连续组成，典型约 3000 个 member；
- 每个 member 属于一个车型，需要使用确定版本的 DBC 解码；
- gzip 解压后执行 CPU-heavy CAN 解析、信号解码和聚合；
- URL 数量极大，系统更关注持续吞吐、成本与可恢复性，而非单文件最低尾延迟。

16.1.1 风险等级

该系统默认按 R2 — Core Production 设计：它处理持续产生的生产数据，结果会被下游分析、诊断或业务流程消费，错误 DBC、重复正式结果或不可追溯输出可能形成长期数据污染。若结果进一步参与车辆安全控制、资金结算或法规判断，应提升至 R3 — High Assurance，增加独立审批、职责分离和持续证据漂移检测。

16.1.2 Gate 映射

评审门禁 本案例对应内容	
Gate 0	问题、Goals、Non-goals 与风险等级
Gate 1	Workload、SLO、约束与关键假设
Gate 2	Critical Properties 与优先级
Gate 3	Source/Operation/Result Identity、状态权威与 DBC 绑定

评审门禁	本案例对应内容
Gate 4	失败、提交点、ACK 顺序与恢复
Gate 5	数据面、控制面、调度、快慢路径与运行模式
Gate 6	有界资源、容量、恢复能力和单位经济性
Gate 7	契约、安全、演进、运维与语言决策
Gate 8	Verification & Evidence Matrix
Gate 9	Change Envelope、停止条件与生产裁决

16.2 Goals / Non-goals

16.2.1 Goals

- 1. 高吞吐、可水平扩展；
- 2. 内存、任务、连接、输出和重试全部有界；
- 3. Kafka 重复投递和 worker crash 不产生重复正式结果；
- 4. DBC 身份和版本可追踪，未知或不匹配时不猜测；
- 5. partial output 不作为 completed result 暴露；
- 6. 可重放、可审计、可恢复；
- 7. CPU pipeline 可测量并按瓶颈优化。

16.2.2 Non-goals

- 1. 不保证全局文件顺序；
- 2. 不追求每个 member 的独立超低延迟；
- 3. 不在第一阶段实现跨区域 active-active；
- 4. 不通过无限并发隐藏对象存储或 DBC Registry 变慢；
- 5. 不把“最新 DBC”作为隐式处理语义。

16.3 Workload、SLO、约束与假设

16.3.1 工作负载模型

正式设计必须从生产采样建立分布，而不是只记录均值。至少测量：

```
files / second: average, p95, peak, burst duration
compressed bytes / file: p50, p95, p99, hard maximum
members / file: p50, p95, p99, hard maximum
compressed bytes / member and inflate ratio
vehicle-model / DBC skew
```

CPU seconds / member and output amplification
object-store, registry and result-store latency distribution
corrupt / unsupported / unknown-model ratio
replay and recovery workload

设计容量必须同时覆盖持续新流量、故障降容和积压恢复；仅验证稳态平均负载不足以支持生产结论。

16.3.2 初始 SLO 示例

维度	示例目标	说明
端到端完成延迟	99% 的受支持对象在目标窗口内正式提交	从 Kafka 可见到 completion manifest
数据完整性	重复正式结果 0；未标记 partial completion 0	硬性质，不以延迟换取
DBC 正确性	正式结果的 DBC identity/digest 覆盖率 100%	未知必须隔离
可恢复性	N-1 条件下在约定时间清空积压	必须包含持续新流量
资源安全	RSS、临时磁盘、连接和队列始终低于硬上限	达到上限时停止接纳
可追溯性	任意正式结果可追溯 source、pipeline、config、binding 与 DBC digest	作为发布与审计条件

数值必须绑定具体 workload corpus、节点规格、依赖版本和成本边界。没有这些限定的“吞吐”或“p99”不构成可复用证据。

16.3.3 关键约束

- 源对象必须可通过不可变版本或内容摘要稳定寻址；
- 单文件当前合同上限为 40 MiB，但 admission 仍按 bytes 而非文件数控制；
- Kafka 提供至少一次投递语义，系统必须吸收重复；
- 对象、member、车型和 DBC 分布可能偏斜；
- CPU-heavy 解码不得占用异步 I/O executor；
- 生产结果只在权威 completion manifest 提交后可见；
- DBC Registry 和 Result Store 可能超时、降级或短时不可用；
- 技术选型受既有库、团队生产经验、诊断能力和 on-call 责任约束。

16.3.4 关键假设登记示例

ID	假设	运行信号	失效条件	响应
A-01	单对象不超过生产合同上限	object_size	任意超限	admission reject； 复审 producer contract
A-02	member 数和 inflate ratio 处于建模范围	members/object、 inflate ratio	超阈值或分布漂移	quarantine；容量 与格式复审
A-03	对象版本读取稳定	version/checksum mismatch	任意不一致	停止处理；重新读取权威版本
A-04	同版本 DBC artifact 内容不可变	digest mismatch	任意不一致	fail closed；触发 Registry 事故
A-05	Result Store 的 conditional create 语义可靠	contract/fault test	语义或实现变化	阻断发布；重新设计提交协议
A-06	当前“整对象下载”在成本与内存上优于细粒度 range	benchmark、 request amplification	对象上限/延迟目标/ 存储计费变化	重新比较 streaming/range 方案

假设必须有 Owner、复审触发和证据状态。失效假设不得继续作为已批准架构的隐含前提。

16.4 关键性质

16.4.1 CP-01 — 单一正式完成结果

Identity: object_version + pipeline_semantic_version
Guarantee: Result Store 最多存在一个正式 completion manifest。
Mechanism: staged output + create-only/conditional completion manifest。
Allowed degradation: 重复执行返回 ALREADY_COMMITTED；冲突进入 reconciliation。

16.4.2 CP-02 — DBC 可追踪且不猜测

每个正式输出必须包含 vehicle_model、dbc_id、dbc_version、 dbc_content_digest 与 pipeline_semantic_version。
UNKNOWN_MODEL / DBC_NOT_FOUND / VERSION_MISMATCH

必须进入显式隔离状态，不得回退到 latest 或相似车型。

16.4.3 CP-03 — Partial output 不可见为正式完成

成员级输出可以暂存，但只有 completion manifest 提交后，下游才将该对象视为完整正式结果。

16.4.4 CP-04 — 资源有界

inflight compressed bytes、active files、CPU jobs、output bytes、open connections、retry budget 和 reconciliation backlog 均有硬上限。

16.4.5 CP-05 — 已接纳任务有显式终态

COMMITTED / QUARANTINED / PERMANENTLY_REJECTED / RECONCILIATION_REQUIRED

不得静默丢失。

16.5 状态与所有权

State	Authority	Identity	Owner	Recovery
Discovery offset	Kafka consumer group	topic/partition/offset	Processing Platform	Kafka
Source bytes	Object Storage	object URI + immutable version	Data Ingestion	re-read exact version
DBC artifact	DBC Registry	dbc_id + version + digest	Vehicle Data Platform	immutable artifact store
Model→DBC binding	Versioned Binding Registry	binding version + model	Vehicle Data Platform	versioned snapshot
Staged outputs	Result Store staging namespace	run_id + member identity	Processing Platform	delete/reconcile
Completion manifest	Result Store	object version + pipeline version	Processing Platform	conditional read/commit

State	Authority	Identity	Owner	Recovery
Runtime queues	Worker process	ephemeral	Runtime Owner	rebuild from durable state
Verification evidence	Evidence Store	evidence_id + subject digest	Assurance Owner	immutable retained record

16.5.1 稳定身份

```
SourceIdentity
= canonical object URI
+ immutable object version or source checksum

PipelineSemanticVersion
= processor version
+ output contract version
+ semantic configuration digest
+ DBC binding policy / snapshot version

OperationIdentity
= SourceIdentity + PipelineSemanticVersion

ResultIdentity
= OperationIdentity + output partition or range identity
```

Kafka offset、Worker 实例、临时文件路径和本次启动时间都是执行身份，不能替代业务幂等身份。重新处理同一源对象但采用不同语义版本时，应形成新的 OperationIdentity，而不是覆盖旧结果。

16.6 DBC 绑定语义

不得在 worker 执行时隐式查询“当前最新 DBC”。推荐在接纳或调度阶段解析出不可变绑定：

```
DecodeBinding
vehicle_model
dbc_id
dbc_version
dbc_content_digest
binding_policy_version
effective_time_basis
```

effective_time_basis 必须明确是采集时间、文件产生时间、任务接纳时间还是显式版本。否则重放相同源数据可能产生不同结果。

数据面只消费不可变 binding 和 DBC artifact；Binding Registry 不可用时，任务可以暂停或隔离，但不能猜测。

16.7 推荐逻辑架构

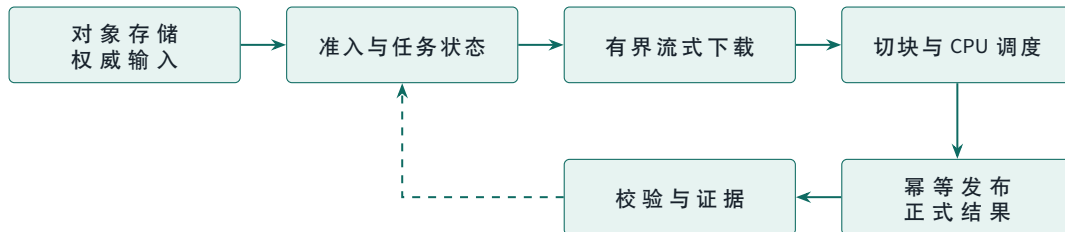


图 2 CAN 案例的有界处理与正式结果发布路径

```
Kafka
-> Admission / Dispatcher
  -> Object Fetchers (async I/O, bounded byte permits)
    -> File Buffer / Bounded Spool
      -> Framer
        -> CPU Work Scheduler
          -> Decompress + Parse + DBC Decode
            -> Bounded Aggregation / Output
              -> Staging Store
                -> Conditional Completion Manifest
                  -> Kafka ACK

Control Plane:
  pipeline semantic version
  DBC binding snapshot
  resource limits
  retry / quarantine policy

Management Plane:
  deploy / drain / replay / reconcile / emergency actions

Assurance Plane:
  contract tests / load tests / fault injection / evidence / gates
```

16.8 为什么默认避免“两遍偏移预扫描”

方案二“先下载并记录所有 gzip 偏移，再交给下游线程”是否有价值，取决于外层格式是否提供可信的压缩长度：

1. 若业务头包含经过校验的 **compressed_length**，Framer 可以低成本单遍定位 member 边界；此时记录 offset table 有利于并行调度、重试和局部定位。

2. 若没有可信长度，gzip/deflate 本身没有可直接跳过到末尾的固定长度字段；发现 member 结束通常需要实际解析压缩流。此时“预扫描”可能等价于先做一遍解压/解析，再由下游重复工作，得不偿失。

因此推荐：

有可靠外层长度：

```
single download -> cheap framing -> parallel member work
```

无可靠外层长度：

```
single download -> one-pass decompress/framing -> decode or fused handoff
```

不得把无法证明正确的 magic-byte 搜索作为 gzip 边界算法。

16.9 下载与缓冲策略

由于单文件硬上限为 40 MiB，默认优先考虑：

完整对象单次下载到 pooled buffer

- + 全局 inflight-byte permits
- + 单遍 framing
- + 不重复读取对象存储

这通常比为 3000 个小 member 发起大量 Range 请求更简单、更经济，也能避免对象存储请求放大。其安全前提是全局按 bytes 限制在途文件，而不是只限制文件数。

若实际吞吐使内存预算不可接受，可选择 bounded spool / mmap 临时文件，但必须增加临时磁盘上限、清理、崩溃恢复和 I/O 模型；不能把 spill 当作免费内存。

16.10 CPU 调度与局部性

推荐分离：

- 异步 I/O executor：对象下载和外部存储请求；
- 固定 CPU worker pool：解压、解析、DBC decode；
- 有界输出 executor：序列化与结果写入。

通常应优先融合 decompress + parse + DBC decode，避免把大块未压缩中间数据跨队列传递。若基准表明解压与 decode 的瓶颈、向量化或库约束明显不同，再拆分独立 CPU stage。

调度可以采用：

```
bounded global ready queue
```

```

+ per-worker local deque / work stealing
+ DBC/model locality hint
+ fairness and starvation bound

```

不要为每个车型永久创建无界队列。局部性优化必须服从资源上界与租户公平。

批大小不应拍脑袋固定。应通过基准在以下目标间权衡：

- 调度开销；
- DBC cache locality；
- cancellation 粒度；
- 输出聚合成本；
- 公平性和尾延迟。

16.11 幂等与提交协议

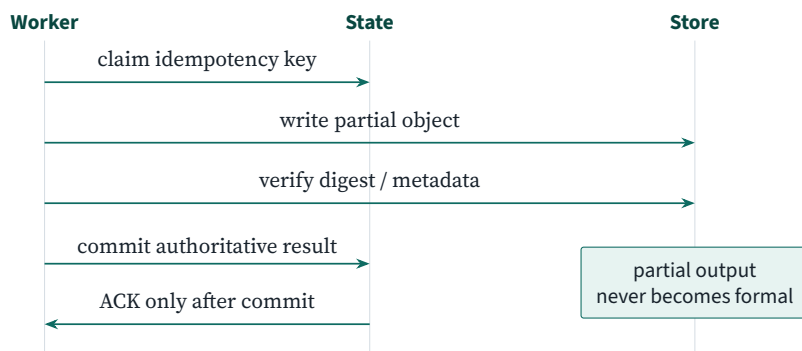


图3 幂等键、partial output 与权威提交的顺序约束

推荐对象级正式提交：

1. 读取 Kafka task, 解析 immutable source identity。
2. 查询 completion manifest; 已存在则验证语义版本后跳过或返回已完成。
3. 下载并处理所有 member, 输出写入 run_id staging namespace。
4. 校验 member count、checksum、error policy 和 output summary。
5. 使用 idempotency key 条件创建 completion manifest。
6. 创建成功：对象正式完成，再提交 Kafka offset。
7. 冲突：读取既有 manifest; 相同语义则清理 staging, 返回 ALREADY_COMMITTED; 不同语义则进入 RECONCILIATION_REQUIRED。

示例幂等身份：

```

hash(
    source_object_uri,
    source_object_version,
    pipeline_semantic_version,

```

```
    output_contract_version
)
```

若业务允许 **member** 级独立可见，可把同一模式下沉到 **member identity**，但会显著增加状态数量、提交放大和 **reconciliation** 复杂度，必须通过业务价值证明。

16.12 有界资源示例

```
max_active_files           = derived from byte budget
max_inflight_source_bytes = B_source
max_cpu_jobs               = near measured CPU saturation point
max_queued_compressed_bytes= B_work
max_output_buffer_bytes    = B_output
max_object_connections     = object-store quota aware
max_retry_attempts        = classified by error
max_retry_time             = within end-to-end deadline
max_reconciliation_backlog = explicit operational bound
```

准入应同时受 **source bytes**、**CPU permits** 和 **output pressure** 约束。只按 **Kafka message** 数量暂停不足以控制大小差异和 **CPU** 成本差异。

16.13 容量模型

定义：

```
lambda_f = files / second
S_c      = average compressed bytes / file
N_m      = members / file
t_d      = CPU seconds / member for decompress + decode
R_o      = output bytes / input byte
```

则：

```
network input demand = lambda_f x S_c
member rate          = lambda_f x N_m
CPU core lower bound = lambda_f x N_m x t_d
output bandwidth     = lambda_f x S_c x R_o
```

实际 **CPU** 预算还要加入 **framing**、聚合、序列化、缓存 **miss**、调度开销和 **headroom**。

示例：

```
lambda_f = 25 files/s
S_c      = 20 MiB/file
N_m      = 3000

t_d      = 80 microseconds/member

input bandwidth ~= 500 MiB/s
member rate     = 75,000/s
CPU lower bound = 6 CPU-seconds/s ~= 6 fully utilized cores
```

若目标利用率为 60%–70%，并考虑恢复和抖动，实际配置显著高于理论 6 核。最终数值必须由代表性语料端到端基准确定。

16.13.1 故障降容与积压恢复

仅证明正常状态下 $\text{service_rate} > \text{arrival_rate}$ 不足以支持高可用结论。还必须验证：

```
recovery_service_rate > live_arrival_rate
catch_up_time ~= backlog / (recovery_service_rate - live_arrival_rate)
```

N-1、依赖恢复、重放和 **rolling upgrade** 都可能与实时流量竞争相同资源。恢复速率、在线流量保护比例和最大 backlog age 必须显式建模。

16.13.2 单位经济性

至少跟踪：

```
cost / GiB source data
cost / million gzip members
cost / vehicle-hour
cost of replay
cost of quarantine and reconciliation
cost of retained evidence
```

性能优化只有在满足关键性质、SLO 和成本边界后才有意义。更高峰值吞吐并不自动优于更稳定、成本更低且恢复能力更强的实现。

16.14 失败与恢复矩阵节选

Failure	Partial Effect	Decision / Recovery
Kafka duplicate	任务可能再次执行	completion manifest 吸收重复
Download timeout	对象可能只下载一部分	丢弃不完整 buffer；按分类重试
Object version changed	输入身份不再稳定	拒绝；要求 immutable version
Corrupt member	部分成员已处理	按合同 quarantine whole object 或显式 partial policy；不得静默完成
Unknown model	无合法 DBC	quarantine；等待 binding 修复后重放
DBC Registry unavailable	binding 无法验证	pause / degraded；不回退猜测
Worker crash before manifest	staging 可能存在	重跑；staging 按 run_id 清理或回收
Manifest committed, ACK failed	正式结果已存在	重投后读取 manifest 并 ACK
Result Store conflict	另一执行已提交	比较语义身份；相同跳过，不同 reconcile
Output slowdown	output queue 增长	背压到 CPU，再到 fetch/Kafka pause
Memory pressure	byte permits 接近上限	停止接纳；不得继续扩大队列

16.15 关键可观测性

```
files_discovered / accepted / committed / quarantined
source_bytes_inflight
active_files
members_processed_rate
fetch / framing / decode / commit service time
per-stage queue depth and queue age
CPU worker utilization and steal rate
dbc_cache_hit / miss / unknown / version_mismatch
retry amplification ratio
staging_age and orphan_staging_count
completion_conflict_rate
end_to_end_lag
recovery_backlog_age
cost_per_GiB / cost_per_vehicle_hour
```

指标应使用稳定业务身份关联 source、run、pipeline version、DBC version 和 completion manifest；避免把高基数原始标签无控制地写入所有指标系统。

16.16 验证计划

Property	Verification
单一正式结果	重复投递、commit/ACK crash、rebalance、并发执行测试
Partial 不可见	在每个阶段注入 crash，验证无 completion manifest 时下游不可见
DBC 不猜测	unknown model、registry timeout、version mismatch 负向测试
资源有界	2× 峰值、持续下游变慢、超大合法文件和混合大小压力测试
背压闭合	人为降低 output/DBC/object store，验证压力传播和内存稳定
恢复能力	积压回放、节点 N-1、进程反复崩溃和 orphan staging 回收
容量模型	代表性语料端到端基准，比较预测与实际误差
版本追踪	对任意结果追溯 source、pipeline、binding、DBC 和 config digest

16.17 Rust 与 C++23 的正确决策位置

两者都能实现该流水线。决策应基于：

- 现有 DBC/压缩/解析库与复用成本；
- 团队熟练度、招聘和 on-call 能力；
- 内存安全与 unsafe/FFI 边界；
- profiling、sanitizer、fuzzing、构建和供应链能力；
- CPU 性能、编译时间和部署目标；
- 长期维护与迁移成本。

推荐用同一代表性语料实现窄范围原型，比较：吞吐、p99、RSS、CPU、崩溃注入、开发复杂度和可观测性，而不是用语言声誉替代证据。

16.18 Change Envelope 与示例裁决

首次生产暴露或重大语义变更应定义受控边界：

```
population:
  selected vehicle models / tenants / object prefixes

exposure:
  max files, max source bytes, max duration, max concurrency
```

```
preconditions:
  required evidence IDs, DBC snapshot, artifact/config digests

success criteria:
  correctness, resource, latency, recovery and cost thresholds

stop conditions:
  duplicate formal result
  DBC digest mismatch
  unbounded queue / RSS growth
  queue age or retry amplification beyond threshold
  missing provenance or evidence binding

recovery:
  stop admission, drain, quarantine, reconcile, forward repair
```

建议先使用 **shadow** 或非权威输出验证 **framing**、**DBC**、**lineage** 和容量；样本必须包含大型对象、热点车型、损坏输入和依赖抖动，而不是只选最干净流量。

示例裁决：

```
Gate 0-5: GO
Gate 6: CONDITIONAL_GO
  scope: verified node specification and <= 2x measured peak workload
  condition: complete N-1 catch-up benchmark
  expiry: 2026-10-15

Gate 7: GO
Gate 8: GO for the declared corpus and failure profile
  unknown exotic header variants: QUARANTINE

Gate 9: GO
  exposure: 5% of object bytes
  automatic abort: any hard-property violation or memory-bound breach
```

这类裁决明确区分已证明范围、剩余未知、临时条件和撤销触发，比“整体看起来可以上线”更可审计。

Production Readiness Checklist

17.1 问题与语义

- ☐ Problem、Goals、Non-goals 和成功标准可检验；
- ☐ 风险等级与失败后果有依据；
- ☐ Safety、Liveness、Durability、SLO 和 Assumption 已分类；
- ☐ 关键性质具有身份、作用域、时间语义、Owner 和违反后果；
- ☐ 性质冲突和允许降级已预定义；
- ☐ 所有阻断级 **UNKNOWN** 已解决或明确拒绝放行。

17.2 状态与边界

- ☐ 权威状态和稳定身份明确；
- ☐ Mutation Authority、Technical Owner、Operational Owner 明确；
- ☐ 缓存、派生状态和副本不会形成隐式多权威；
- ☐ Boundary Contract 覆盖成功、错误、重复、乱序、超时和容量假设；
- ☐ 上游 Guarantee 满足下游 Assumption；
- ☐ 配置、策略、模型和参考数据按语义版本化。

17.3 失败与恢复

- ☐ 部分成功、结果未知和重复执行有明确收敛；
- ☐ ACK 与业务完成语义一致；
- ☐ 幂等身份稳定，原子提交或去重机制明确；
- ☐ Retry 只用于分类后的可重试错误，并有共享预算；
- ☐ Normal、Degraded、Recovery、Quarantine、Emergency 模式明确；
- ☐ 回滚、forward-recovery、reconciliation 和灾难恢复可执行；
- ☐ RPO、RTO 和恢复积压目标经过演练或证据支持。

17.4 资源、性能与成本

- ☐ 所有可增长资源有硬上限；
- ☐ 队列容量单位、满载行为、背压和公平性明确；
- ☐ 峰值、N-1、扩容延迟和恢复容量经过计算；
- ☐ `retry amplification` 和控制回路相互作用经过评审；
- ☐ 端到端基准使用代表性语料和目标环境；
- ☐ 单位经济性至少达到 C1 粗估；
- ☐ 新增复杂度具有 Owner、必要性和回收条件。

17.5 安全、运维与演进

- ☐ Trust Boundary、最小权限和高风险变更权威明确；
- ☐ `secret`、证书、`artifact` 和 `dependency provenance` 可追踪；
- ☐ Schema / protocol 兼容与混合版本行为经过验证；
- ☐ `deploy`、`drain`、`shutdown`、`upgrade`、`migration`、`replay` 有 Runbook；
- ☐ `break-glass` 有范围、时限、审计与补偿复审；
- ☐ 告警对应明确行动，而不是仅通知；
- ☐ Operational Owner 已接受 on-call 和恢复责任。

17.6 验证与证据

- ☐ 每项关键性质映射到机制、验证和证据；
- ☐ 证据绑定当前制品、配置、环境、语料和故障档案；
- ☐ 证据覆盖、限制、状态和失效规则明确；
- ☐ Assurance Plane 变更本身受保护；
- ☐ 例外和风险接受有到期时间；
- ☐ Change Envelope 定义暴露、成功、停止和恢复条件；
- ☐ 运行漂移能够使相关结论降级为 **UNKNOWN**。

最小 19 问设计清单

在时间有限的评审中，至少回答以下 19 个问题：

1. 系统解决什么具体问题，什么明确不做？
2. 成功如何衡量，失败会造成什么后果？
3. 真实工作负载、尾部分布、突发和增长是什么？
4. 技术、组织、经济、安全、合规和时间约束是什么？
5. 哪些关键假设尚未证明，何时会失效？
6. 哪些 Safety、Liveness、Durability 和 SLO 必须成立？
7. 关键性质冲突时，哪个优先，谁裁决？
8. 每份权威状态的身份、存储、Mutation Authority 和 Owner 是谁？
9. 关键边界的 Assumption、Guarantee 和违反行为是什么？
10. 重复、乱序、超时、部分成功和 crash 时会发生什么？
11. 业务完成点、ACK 顺序、幂等身份和冲突语义是什么？
12. 一致性选择如何由业务性质推导，而不是由组件默认决定？
13. 所有队列、缓存、任务、连接、重试和临时状态是否有界？
14. 背压能否传播到可控上游，持续过载时如何处置？
15. 峰值、故障降容、积压恢复和单位成本是否可计算？
16. 正常、降级、恢复、隔离和紧急模式分别允许什么？
17. 系统如何发布、排空、升级、迁移、回滚、重放和灾难恢复？
18. 每项关键主张用什么机制、验证和当前证据支持？
19. 假设、证据或环境失效后，谁把结论降为 **UNKNOWN** 并触发复审？

任何高后果问题回答为“后续再看”，都应被视为显式风险，而不是默认通过。

团队成熟度与采用路线

等级	典型状态	下一步
L0 — Component-driven	先选技术，缺少问题和状态模型	强制 One-page Brief 与 Non-goals
L1 — Functional	正常路径可用，失败与 Owner 模糊	建立 Property、State、Failure Register
L2 — Bounded	资源有界，有基本恢复	增加容量、运行模式和边界契约
L3 — Operable	SLO、Owner、发布和恢复明确	建立 Evidence Matrix 与生产门禁
L4 — Evidence-driven	关键主张有可追踪证据	关联假设、环境漂移和证据时效
L5 — Adaptive Assurance	结论随漂移自动失效，事故形成保 证增量	持续防止 Goodhart 化并回收复 杂度

推荐采用顺序：

- 先统一术语和最小模板

 - > 在一个高价值项目试点
 - > 用真实评审和事故校准
 - > 将高确定性机械规则编码进 CI
 - > 保留人工审查处理未编码属性
 - > 建立成本、证据与漂移平台能力
 - > 定期删除低价值门禁和长期例外

不要试图通过一次性发布完整标准让团队“直接达到 L4”。流程厚度必须与现有能力和风险匹配，但关键状态、身份和认识边界必须从第一天保持诚实。

文档与标准维护

20.1 版本规则

- 原则、术语或门禁语义发生不兼容变化：提升 Major；
- 新增兼容的模板、场景或要求：提升 Minor；
- 文字澄清、排版和不改变语义的修正：提升 Patch；
- 本文与方法论文档使用统一 `suite version`，但允许独立 `patch`。

20.2 复审触发

以下任一事件触发复审：

- 重大事故或近失事件；
- 关键假设失效；
- 新合规、安全或业务硬约束；
- 工作负载、成本或故障域显著变化；
- 新的分布式边界、权威状态或特权主体；
- 现有门禁频繁被例外绕过；
- 证据与生产漂移长期无法关联；
- 至少每 12 个月一次基线复审，高风险系统应更短。

20.3 单一事实来源

本 Markdown 文件是执行参考的唯一权威源。任何表单、门户、CI 配置或导出视图必须能够追溯到本文版本；不得维护无法同步的平行“精简版标准”。实际项目可以实例化模板，但不得静默改变规范语义。

最终操作原则

评审的目的不是证明设计者足够聪明，也不是让所有检查变绿，而是在代码和流量扩大之前，尽早暴露状态、失败、资源、权限、恢复和证据链中的高代价错误。最好的门禁会阻止不可接受风险，同时让低风险、可逆决策保持快速。

20.4 两篇文档的最终边界

ESD-METHOD-001

定义：为什么这样设计、核心术语是什么、能够证明到什么范围

ESD-REFERENCE-001

执行：按什么阶段工作、提交什么制品、如何评审、裁决和持续维护

领域 **Profile** 可以收紧要求或按风险裁剪执行深度，但不得静默改变方法论中的关键术语、认识边界、状态权威和 **UNKNOWN** 语义。

20.5 结论

高质量 **Architecture Review** 的目标不是批准一张架构图，而是阻止未经定义的状态、失败、权限、资源和未知进入生产。一个完成的设计应留下可追踪的推理链：

Problem

- > Constraints / Assumptions / Risk
- > Critical Properties
- > State / Identity / Ownership
- > Failure / Consistency / Recovery
- > Architecture / Resource Control
- > Capacity / Economics / Evolution
- > Verification / Evidence
- > Controlled Change
- > Runtime Learning

当团队能够在任意环节诚实回答“我们知道什么、为什么相信、覆盖到哪里、什么时候失效”，系统设计才从经验与审美上升为可治理的工程实践。

优秀系统设计与工程保证标准

ESD-REFERENCE-001 · Version 1.1.0

Generated from the authoritative Markdown source.